// Sorting - 4

Agenda:

1. Inbuilt Sorting functions
2. Comparators Basics
3. Using Lambda with Comparator
4. factors Sort
5. Largest Number
6. Sorting array of Objects

① Inbuilt Sorting functions

Both use T.C = $O(N \log N)$

ⓐ **Arrays.sort()**

```
int [] arr = {5,2,8,1,9};
Arrays.sort(arr);
Sopln (Arrays.toString(arr)); → 1 2 5 8 9
```

ⓑ **Collections.sort()**

```
ArrayList<Integer> list = new ArrayList<>(Arrays.asList(5,2,8,1,9));
Collections.sort(list);
Sopln (list);
```

⇒ **Sort in Descending Order**

- Collections.sort(list, Collections.reverseOrder());
- Arrays.sort(arr, Collections.reverseOrder());

Comparators → Help define custom Order

- Using Lambda Expressions: → only for Objects (Integer ✓, int ✗)

Case: (a, b) if a is before b

Collections.sort(list, (a, b) -> **return Value**);

if
- return < 0 → a comes before b
- return = 0 → a = b
- return > 0 → a comes after b

ex.

```
int fx = countFactors(V1);
int fy = countFactors(V2);
return fx - fy;
```

- No. with fewer factors come first
- No. with more factors come later.

(ascending order by no. of factors)

(refer VS code stored ex.) (class made)

list → { Zara, Alex, Bob, Alex }

- for Strings
  (sorting array of objects of class)

~~ex. humans.sort((h1, h2) → h1.getName().compareTo(h2.getName()))~~

→ if comes before, put before (in dictionary)

ex: humans.sort((h1, h2) → h1.getName().compareTo(h2.getName()));

ex: human.sort((h1, h2) → h1.getAge() - h2.getAge());

Q. **Sorting by factors**

↳ Sort based on ~~asce~~ count of factors. If 2 numbers have same no. of factors, sort them by their magnitude.
(in ascending order)

λ implementation:

```
public ArrayList<Integer> solve (ArrayList<Integer> A){
        Collections.sort (A, (v1, v2) -> {
                int fx = countFactors (v1);
                int fy = countFactors (v2);
                if (fx != fy) { return fx - fy; }
                else { return v1 - v2; }
        });
        return A;
}

private int countFactors (int n){
        int count = 0;
        for (int i = 1; i*i <= n; i++){
                if (n % i == 0){
                        count++;
                        if (i*i != n) count++;
                }
        }
        return count;
}
```

Q. **Largest Number**

↳ A list of non-negative ~~num~~ integers is given, arrange them so they form the largest possible number.

Ex. for [10, 2] → ans. is "210"

Code →

```
public String largestNumber(ArrayList<Integer> A){
Collections.sort (A, (a,b) -> {        // sort no. based on concatenation logic
    String s1 = String.valueOf(A) + String.valueOf(b);
    String s2 = String.valueOf(A) + String.valueOf(a);
    return s2.compareTo(s1);           // Reverse order to get the largest combo first
});
if (A.get(0) == 0) return "0";         // special case: if the largest no. is 0 [eg: [0,0]]
StringBuilder ans = new StringBuilder();
for (int num : A) {
    ans.append(num);
}
return ans.toString();
}
```

Q. **Sorting by Digit Sum**

```
Arrays.sort(nums, (a,b) -> recSum(a) - recSum(b));
Sopln(Arrays.toString(nums));
}
static int recSum(int n){
    if (n==0) return 0;
    return (n%10) + recSum(n/10);
}
```

Comparators work only with objects, not (Primitives).

- ✗ int → primitive
- ✗ int[] → object → but elements primitive
- ✓ int[][] → array of int[] object → element are object

☆ ~~Gps~~

Gps Hotel Problem (Sorting-4)

```
public int [][] solve (int[][] A, int B){
    Arrays.sort (A, (P1, P2) -> {
        long d1 = (long) P1[0] * P1[0] + (long) P1[1] * P1[1]
        long d2 = (long) P2[0] * P2[0] + (long) P2[1] * P2[1];
        return Long.compare (d1, d2);
    });
```

Sort points by distance of origin

ideal way to compare long values

```
    int [][] ans = new int [B][2];
    for (int i=0; i<B; i++){
        ans[i][0] = A[i][0];
        ans[i][1] = A[i][1];
    }
    return ans;
}
}
```

Arraylist approach:

```
public ArrayList < Arraylist < Integer >> solve (ArrayList < Arraylist < Integer >> A, int B){
    Collections.sort (A, (Arraylist <Integer> a, Arraylist < Integer> b) -> {
        double x = Mathsqrt ( --- same as before --);
        double y = Mathsqrt ( --- same as before --);
        return (int) ((x-y) (Math.abs (x-y)));
    });
    ArrayList <Arraylist <Integer >> ans = new Arraylist <>();
    for (int i=0; i<B; i++ ){
        ans.add (A.get(i));
    }
    return ans;
```